Messkonzept: Zuverlässigkeit einer verteilten Datenkette mit MQTT

Versuchsaufbau, Fehlerorte, Messgrößen und Architekturvarianten

Erlind Sejdiu · Stand: 3. November 2026

1 Versuchsaufbau

Abbildung 1 zeigt die bestehende Architektur als verteiltes System. Ein Reader liest die Maschinen aus und schreibt in eine Datenbank. Ein Übertragungsmodul [Name] überträgt die Daten nach Cancom [Art des Systems klären], von dort gelangen sie zum API-Server, der sie mit MQTTnet an EIS, Hallenspiegel und künftig die Feinplanung verteilt. Die Kette wird in einer isolierten Testumgebung nachgebildet [welche Komponenten können im Original betrieben werden, welche werden nachgebaut?]. An die Stelle der Maschinen tritt ein Simulator, der jede Meldung mit einer fortlaufenden Sequenznummer und einem Zeitstempel versieht. Zwischen den Stufen liegt jeweils ein Toxiproxy, über den Netzwerkfehler eingebaut werden. Der Testrunner (xUnit) steuert Last und Fehler und sammelt die Messdaten.

Rote Kreise markieren die Fehlerorte 1–6, blaue Kästen die Messpunkte M1–M6. Die Messpunkte liegen jeweils am Ausgang einer Stufe, sodass sich Verluste einer bestimmten Übergabe zuordnen lassen, ohne in die Komponenten selbst einzugreifen.

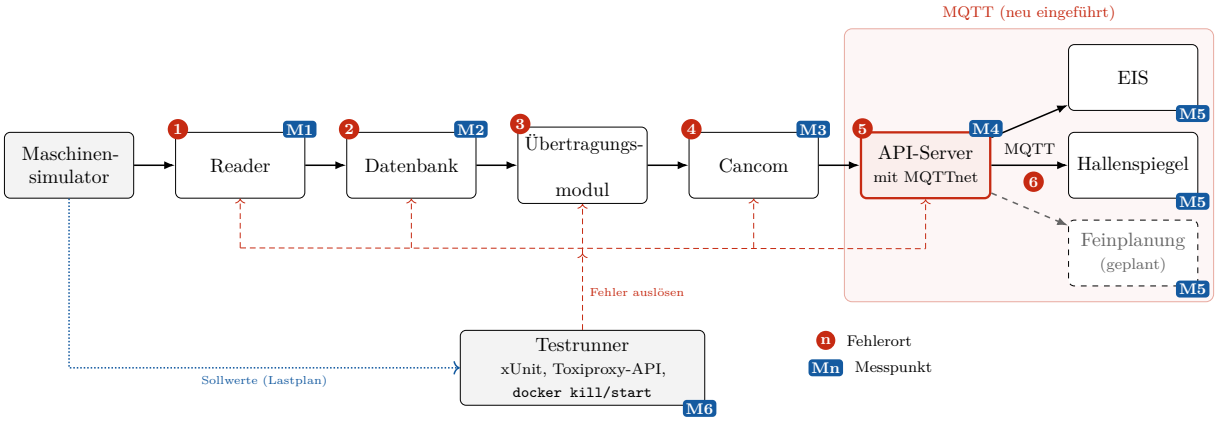


Abbildung 1: Bestehende Architektur als verteilte Datenkette mit Fehlerorten und Messpunkten. MQTT wird nur auf der letzten Strecke eingesetzt und bildet dort einen Stern um den API-Server.

1.1 Fehlerorte

Ort	Stufe	Eingebauter Fehler	Werkzeug
1	Reader	Neustart, Verbindung zur Maschine unterbrochen	`docker kill/start`, Toxiproxy
2	Datenbank	nicht erreichbar, langsame Schreibzugriffe	Toxiproxy, `docker pause`
3	Übertragungsmodul	Absturz während einer Übertragung	`docker kill/start`
4	Cancom	nicht erreichbar, verzögerte Antworten	Toxiproxy
5	API-Server mit MQTTnet	harter Absturz, sauberer Neustart	`docker kill/stop`
6	Strecke zu den Verbrauchern	Verbraucher getrennt oder offline	Toxiproxy, `docker stop`

1.2 Messpunkte

Punkt	Ort	Erfasste Rohdaten
M1	Ausgang Reader	Sequenznummer s, Zeitstempel an der Quelle t_{src}, Anzahl ausgelesener Meldungen
M2	Datenbank	gespeicherte Sequenznummern, Schreibzeitpunkt
M3	Cancom	angekommene Sequenznummern, Ankunftszeitpunkt
M4	API-Server	publizierte Sequenznummern, verbundene Clients, gepufferte Nachrichten, CPU, RAM
M5	EIS, Hallenspiegel, Feinplanung	empfangene Sequenznummern in Ankunftsreihenfolge, Empfangszeitpunkt t_{recv}, abgeleiteter Zustand
M6	Testrunner	Sollwerte aus dem Lastplan, Beginn und Ende jedes Fehlers

Alle Komponenten laufen als Container auf einem Host, damit alle Zeitstempel dieselbe Uhr verwenden.

2 Messmethodik

2.1 Vom Qualitätsmodell zur Messung

ISO/IEC 25010 [1] legt fest, welche Teilmerkmale die Zuverlässigkeit ausmachen, aber nicht, wie sie gemessen werden. Die Messung stützt sich deshalb auf ISO/IEC 25023 [2], die je Teilmerkmal Qualitätsmaße mit einer Messfunktion meist der Form $X = A/B$ angibt. Diese Maße setzen voraus, dass sich Ausfälle zählen und Betriebs- und Ausfallzeiten klar abgrenzen lassen. In der Datenkette ist das nicht unmittelbar gegeben: Fällt eine Stufe aus, arbeiten die übrigen weiter, Verbraucher können veraltete oder unvollständige Daten anzeigen, ohne selbst gestört zu sein, und verschiedene Verbraucher können gleichzeitig unterschiedliche Zustände haben. Ein System, das nur entweder läuft oder steht, bildet das nicht ab.

Die Messung geht daher in drei Schritten vor:

1. Für jeden Verbraucher wird zu jedem Zeitpunkt ein Zustand bestimmt. Daraus folgt, was in dieser Arbeit als Ausfall gilt (Abschnitt 2.2).
2. Die Qualitätsmaße aus ISO/IEC 25023 werden auf diese Ausfalldefinition angewendet (Abschnitt 3.1).
3. Ergänzende Messgrößen erfassen, was die Maße der Norm nicht abbilden: Verlust je Stufe, Abweichungen zwischen Verbrauchern und das Zuschalten neuer Verbraucher (Abschnitt 3.2).

2.2 Zustände eines Verbrauchers

Für jeden Verbraucher j (EIS, Hallenspiegel, Feinplanung) wird zu jedem Zeitpunkt t ein Zustand $z_j(t)$ bestimmt. Sind mehrere Bedingungen erfüllt, gilt der weiter unten stehende Zustand.

Zustand	Bedingung
Aktuell	Datenalter $a_j(t) \le a_{\max}$ und alle Meldungen mit $t_{\text{src}} \le t - a_{\max}$ sind angekommen
Veraltet	$a_j(t) > a_{\max}$, aber es fehlt keine Meldung. Die Daten kommen verzögert, aber vollständig an
Lückenhaft	mindestens eine Meldung mit $t_{\text{src}} \le t - a_{\max}$ fehlt. Der angezeigte Zustand weicht vom Sollzustand ab
Getrennt	keine Verbindung zum API-Server bzw. Broker

Als *Ausfall* gilt jeder Übergang eines Verbrauchers aus „Aktuell“ in einen anderen Zustand, als *Ausfallzeit* die Zeit bis zur Rückkehr nach „Aktuell“. Damit werden Teilausfälle erfasst: Ein Verbraucher kann lückenhaft sein, während die anderen aktuell sind. Kommt eine fehlende Meldung später noch an, etwa weil Variante B sie nachholt, kehrt der Verbraucher nach

„Aktuell“ zurück. Was am Ende eines Laufs noch fehlt, geht in den RPO ein.

2.3 Fehlerinjektion und Ablauf eines Laufs

Fehler werden kontrolliert eingebaut, wie es im Chaos Engineering üblich ist [3]: Zuerst wird das Verhalten im Normalbetrieb erfasst, dann ein Fehler ausgelöst und die Abweichung davon gemessen. Jeder Lauf folgt dem Ablauf in Abbildung 2.

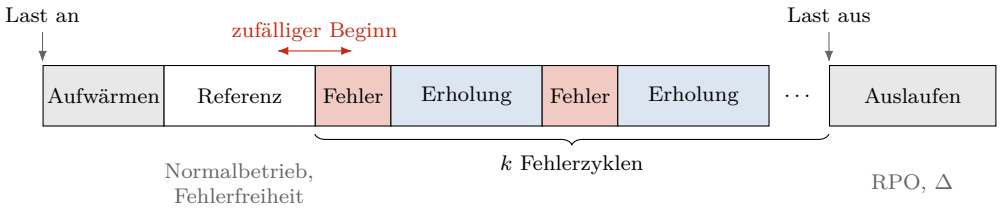


Abbildung 2: Ablauf eines Messlaufs. Die Referenzphase liefert die Vergleichswerte des Normalbetriebs, jeder Fehlerzyklus liefert Fehlertoleranz und RTO, das Auslaufen den endgültigen Verlust.

Der Beginn jedes Fehlers wird zufällig gewählt. Ob ein Fehler folgenlos bleibt, hängt oft davon ab, in welchem Moment er eine Übertragung trifft, etwa ob das Übertragungsmodul vor oder nach der Bestätigung einer Meldung abstürzt. Die Wirkung eines Fehlers ist deshalb eine Wahrscheinlichkeit, die nur über viele Fehlerzyklen geschätzt werden kann. Ein neuer Fehlerzyklus beginnt erst, wenn alle Verbraucher wieder aktuell sind oder eine Höchstdauer [z. B. 120 s] überschritten ist.

2.4 Sollzustand als Vergleichsgrundlage

Der Simulator erzeugt die Meldungen aus einem Lastplan mit festem Startwert für den Zufallsgenerator, sodass jeder Lauf reproduzierbar ist. Der Sollzustand $S_{\text{soll}}(t)$, etwa die Stückzahl je Auftrag, wird aus allen bis t erzeugten Meldungen berechnet. Der Istzustand $S_j(t)$ eines Verbrauchers wird mit derselben Rechenvorschrift aus den an M5 empfangenen Meldungen berechnet. So lassen sich Soll und Ist vergleichen, ohne die interne Logik der Verbraucher zu kennen. Wo ein Verbraucher seinen Zustand über eine Schnittstelle ausgibt, wird dieser zusätzlich abgefragt und mit $S_j(t)$ verglichen [klären, ob EIS und Hallenspiegel ihren Zustand abfragbar machen].

3 Messgrößen

Es seien N die Anzahl erzeugter Meldungen laut Lastplan, R_k die Folge der an Messpunkt k beobachteten Sequenznummern, $|R_k|_{\text{u}}$ die Anzahl verschiedener Sequenznummern darin, m die Anzahl der Verbraucher und T die ausgewertete Beobachtungsdauer.

3.1 Qualitätsmaße nach ISO/IEC 25023

Tabelle 1 ordnet die Zuverlässigkeitsmaße aus ISO/IEC 25023 [2] der Datenkette zu. Die Norm bezieht sich noch auf ISO/IEC 25010:2011, dort heißt die Fehlerfreiheit noch Reife (*maturity*). Ausfall und Ausfallzeit sind wie in Abschnitt 2.2 definiert [Bezeichner und Messfunktionen gegen den Normtext prüfen].

3.2 Messgrößen der Datenkette

Die Maße der Norm fassen einen Lauf in wenigen Zahlen zusammen. Um zu erklären, an welcher Übergabe Meldungen verloren gehen und wie sich die Verbraucher untereinander unterscheiden, kommen die folgenden Messgrößen hinzu. Ergänzend wird der Aufwand jeder Architekturvariante als Leistungseffizienz gemessen, damit die Abwägung sichtbar wird.

Teilmerkmal	**Maß (ID)**	**Messfunktion**	**Anwendung in der Kette**
Fehlerfreiheit	MTBF (RMa-2-G)	Betriebszeit / Anzahl Ausfälle	Dauerlauf F1, nur Ausfälle ohne eingebauten Fehler
	Ausfallrate (RMa-3-G)	Anzahl Ausfälle / Beobachtungsdauer	F1, je Verbraucher und für die Kette
	Testabdeckung (RMa-4-S)	durchgeführte / geplante Szenarien	Szenarien aus Abschnitt 5.1 je Variante
	Fehlerbehebung (RMa-1-G)	behobene / gefundene Fehler	nicht anwendbar, betrifft den Entwicklungsprozess
Verfügbarkeit	Systemverfügbarkeit (RAv-1-G)	Zeit in „Aktuell“ / T	je Verbraucher A_j, für die Kette A_{Kette} = Zeit, in der alle aktuell sind
	Mittlere Ausfallzeit (RAv-2-G)	Ausfallzeit gesamt / Anzahl Ausfälle	je Verbraucher und Szenario
Fehlertoleranz	Ausfallvermeidung (RFt-1-G)	folgenlose / eingebaute Fehler	folgenlos: kein Verbraucher verlässt „Aktuell“
	Redundanz (RFt-2-S)	redundant ausgelegte / alle Komponenten	aus der Architektur je Variante, ohne Messung
	Mittlere Fehlermeldezeit (RFt-3-S)	$\frac{1}{n}\sum_i (t_{\text{meld},i} - t_{\text{fault},i})$	Zeit bis zur ersten Fehlermeldung in Log oder Monitoring [abhängig vom vorhandenen Logging]
Wiederherstellbarkeit	Mittlere Wiederherstellungszeit (RRe-1-G)	$\frac{1}{n}\sum_i \text{RTO}_i$	RTO wie in Abschnitt 3.2, je Fehlerort
	Vollständigkeit der Sicherung (RRe-2-S)	gesicherte / zur Wiederherstellung benötigte Daten	Anteil der beim Fehler unterwegs befindlichen Meldungen, die in einer persistenten Stufe (Datenbank, Broker-Persistenz) vorliegen

Tabelle 1: Zuverlässigkeitsmaße aus ISO/IEC 25023 und ihre Anwendung auf die Datenkette.

Teilmerkmal	Messgröße	Definition	Quelle
Fehlerfreiheit	Duplikatquote	$D = (\lvert R_5\rvert - \lvert R_5\rvert_{\mathrm{u}}) \,/\, N$ im Normalbetrieb	M5 M6
	Umordnungen	$U = \lvert\{\, i : r_{i+1} < r_i \,\}\rvert$ je Maschine beim Verbraucher	M5
	Einbindungszeit	$t_{\mathrm{sync}} - t_{\mathrm{join}}$: vom Verbinden eines neuen Verbrauchers bis sein Zustand dem Sollzustand entspricht (F2)	M5 M6
Verfügbarkeit	Datenalter	$a_j(t) = t - t_{\mathrm{src}}$ des zuletzt bei Verbraucher j angekommenen Werts, als p50, p95, p99	M1 M5
	Mittlere Teilverfügbarkeit	$\bar{A} = \frac{1}{m}\sum_j A_j$, im Unterschied zu A_{Kette}, das nur zählt, wenn alle aktuell sind; $a_{\max}$ [z. B. 5 s, mit ProSeS festlegen]	M5
	Modell der Kette	$A_{\mathrm{Modell}} = \prod_i A_i$ der Stufen bei unabhängigen Ausfällen, Vergleich mit gemessenem A_{Kette}	M1–M5
Fehlertoleranz	Verlustquote	$L = (N - \lvert R_5\rvert_{\mathrm{u}}) \,/\, N$ unter eingebautem Fehler	M5 M6
	Verlust je Stufe	$L_k = \lvert R_{k-1}\rvert_{\mathrm{u}} - \lvert R_k\rvert_{\mathrm{u}}$, zeigt, an welcher Übergabe Meldungen verloren gehen	M1–M5
	Duplikatquote	D unter eingebautem Fehler	M5 M6
	Folgenlos je Fehlerort	$\hat{p}_i$ = Anteil folgenloser Fehler an Ort i (RFt-1-G je Ort), mit Wilson-Konfidenzintervall [4]	M5 M6
	Abweichung zwischen Verbrauchern	$\delta_{jl}(t) = \lvert S_j(t) - S_l(t)\rvert$ und Dauer, in der $\delta_{jl} > 0$, z. B. Hallenspiegel zeigt andere Stückzahl als EIS	M5
Wiederherstellbarkeit	RTO	$t_{\mathrm{rec}} - t_{\mathrm{fault}}$, wobei t_{rec} der erste Zeitpunkt ist, ab dem der Zustand beim Verbraucher wieder dem Sollzustand entspricht; für die Kette das Maximum über alle Verbraucher	M5 M6
	RPO	Anzahl der Meldungen, die nach Abschluss aller Wiederanläufe endgültig fehlen	M5 M6
	Zustandsabweichung	$\Delta = \lvert\,\mathrm{Zustand}_{\mathrm{ist}} - \mathrm{Zustand}_{\mathrm{soll}}\,\rvert$ am Ende des Laufs, je Verbraucher	M5 M6
Aufwand	Latenz	$t_{\mathrm{recv}} - t_{\mathrm{src}}$ von Ende zu Ende und je Stufe, als p50, p95, p99	M1–M5
	Ressourcen	CPU, RAM, Plattenbedarf je Stunde	M4

3.3 RTO und RPO am Beispiel eines Absturzes des API-Servers

Abbildung 3 zeigt schematisch, wie sich RTO und RPO aus dem Verlauf des Zustands beim Hallenspiegel ergeben, hier der Stückzahl eines Auftrags. Während der API-Server ausgefallen ist, bleibt der Wert stehen. Variante A verliert die Meldungen, die während des Ausfalls nicht zugestellt werden können, und läuft danach mit konstantem Abstand zum Sollwert weiter. Variante B holt sie nach dem Wiederanlauf nach und erreicht den Sollzustand zum Zeitpunkt t_{rec}.

4 Architekturvarianten

Verglichen werden drei Varianten [mit ProSeS abstimmen]. Die bestehende Kette ist die Basislinie.

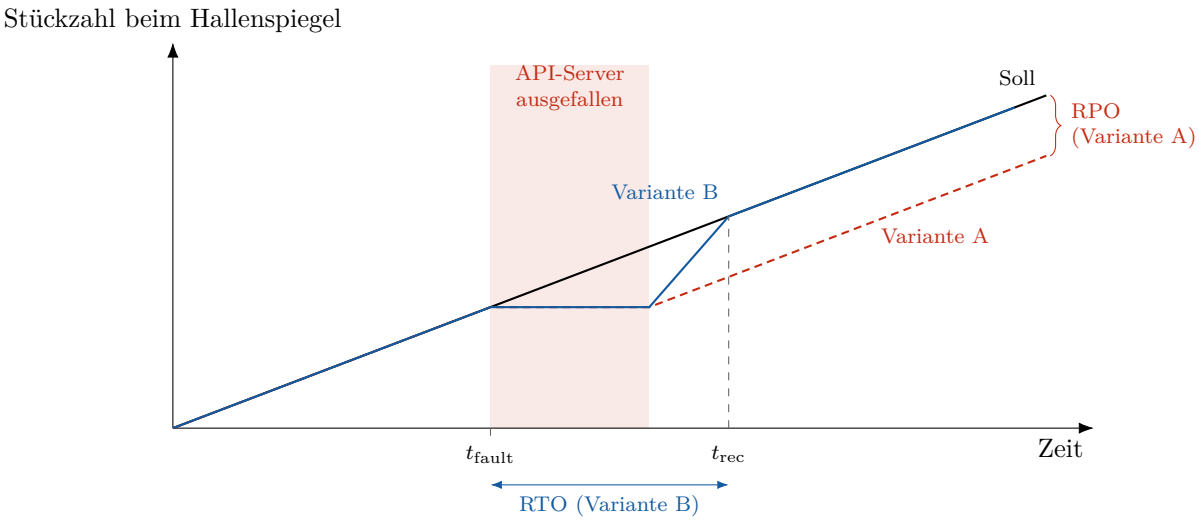


Abbildung 3: Schematischer Verlauf der Stückzahl beim Hallenspiegel bei einem Absturz des API-Servers. Kein Messergebnis, sondern Veranschaulichung der Messgrößen.

Variante	Bezeichnung	Beschreibung
A	Bestehende Kette	Architektur wie in Abbildung 1, unverändert nachgebildet
B	Gehärtete Kette	gleiche Stufen, aber mit Wiederaufsetzpunkt im Übertragungsmodul, idempotenten Übergaben über die Sequenznummer sowie persistenten Sessions und Retained Messages im API-Server
C	Stern	Der Reader veröffentlicht direkt auf einem Broker. Datenbank, Cancom-Anbindung, EIS, Hallenspiegel und Feinplanung sind gleichberechtigte Abonnenten (Abbildung 4)

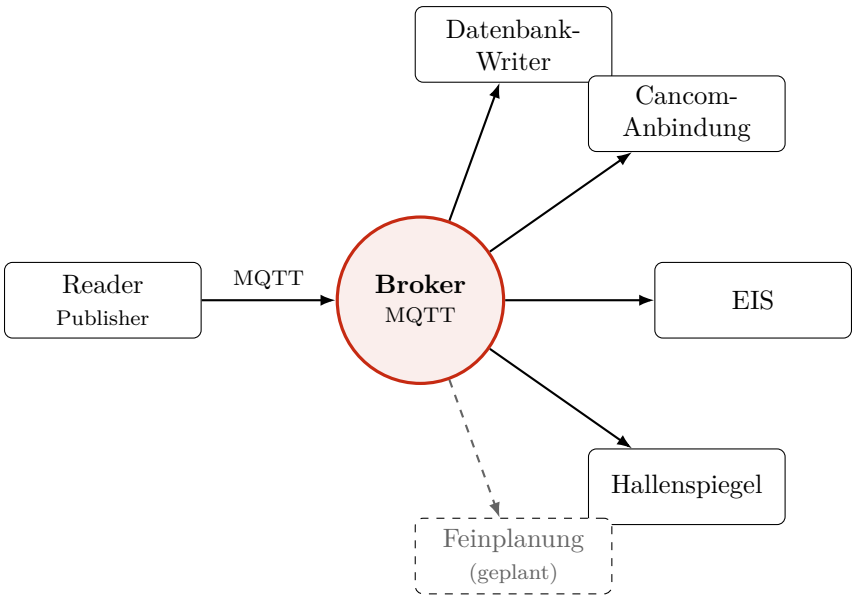


Abbildung 4: Variante C: Sterntopologie. Jeder Verbraucher hängt direkt am Broker und ist nicht mehr von den vorherigen Stufen der Kette abhängig.

Die Varianten stehen für eine Abwägung. Variante B behält die bestehende Architektur bei und sichert jede Übergabe einzeln ab. Variante C verkürzt den Weg zu jedem Verbraucher auf zwei Übergaben, macht den Broker aber zum zentralen Fehlerpunkt und erfordert einen

Umbau.

5 Versuchsplan

5.1 Szenarien

ID	Teilmerkmal	Ort	Szenario
F1	Fehlerfreiheit	–	Dauerlauf 24 h ohne Fehler [Anzahl Maschinen und Takt festlegen]
F2	Fehlerfreiheit	–	Feinplanung wird im laufenden Betrieb als neuer Verbraucher zugeschaltet
V1	Verfügbarkeit	5	Neustart des API-Servers während laufender Fertigung
V2	Verfügbarkeit	6	Hallenspiegel 30 s vom Netz getrennt
T1	Fehlertoleranz	2	Datenbank 30 s nicht erreichbar
T2	Fehlertoleranz	4	Cancom 30 s nicht erreichbar
T3	Fehlertoleranz	3	Übertragungsmodul stürzt während einer Übertragung ab
W1	Wiederherstellbarkeit	5	API-Server hart beendet (`docker kill`) und neu gestartet
W2	Wiederherstellbarkeit	1	Reader während laufender Fertigung neu gestartet
W3	Wiederherstellbarkeit	6	Verbraucher länger offline als die Session-Ablaufzeit

5.2 Faktoren

Faktor	Stufen
Architektur	A: bestehende Kette B: gehärtete Kette C: Stern
QoS (MQTT-Strecken)	0 1 2
Last	[Anzahl Maschinen und Meldungen pro Sekunde, angelehnt an den Produktivbetrieb]
Wiederholungen	[10] Läufe je Kombination mit je [10] Fehlerzyklen, also [100] eingebaute Fehler je Kombination

Nicht jede Kombination ist für jedes Szenario sinnvoll. Je Szenario werden nur die Faktoren variiert, die den eingebauten Fehler betreffen. Die übrigen bleiben auf einem festen Basiswert [Basiswerte festlegen].

6 Umsetzung der Messung

6.1 Nachrichtenformat

Der Simulator versieht jede Meldung mit den folgenden Feldern. Sie müssen alle Stufen bis zum Verbraucher unverändert durchlaufen, damit eine Meldung an jedem Messpunkt wiedererkannt wird [prüfen, ob Übertragungsmodul und Cancom zusätzliche Felder durchreichen, sonst in vorhandenen Feldern codieren].

Feld	Bedeutung
`run_id`	Kennung des Laufs, trennt Daten verschiedener Läufe
`machine_id`	simulierte Maschine
`seq`	fortlaufende Sequenznummer je Maschine, beginnend bei 1
`t_src`	Erzeugungszeitpunkt in Millisekunden
Nutzdaten	fachlicher Inhalt wie im Produktivbetrieb, z. B. Auftrag und Stückzahl

6.2 Messsonden

Jede Sonde schreibt je beobachteter Meldung eine Zeile mit `run_id`, Messpunkt, `machine_id`, `seq`, `t_src` und Beobachtungszeitpunkt t_{obs}. Die Sonden sollen die Komponenten möglichst nicht verändern.

Punkt	Umsetzung	Rückwirkung auf das System
M1	Log des Readers. Ist der Reader nicht instrumentierbar, wird der Verlust zwischen Simulator und Datenbank über M6 und M2 bestimmt	keine
M2	Abfrage der Datenbank nach dem Lauf über Sequenznummer und Schreibzeitpunkt. Fehlt ein Schreibzeitpunkt, wird eine Spalte mit Standardwert ergänzt	keine während des Laufs
M3	Export oder Log von Cancom [Zugriff klären]	offen
M4	passiver Mitleser, der alle Topics abonniert, sowie `docker stats` für CPU und RAM im Sekundentakt	zusätzlicher Client am Broker
M5	Verbraucher-Nachbildungen mit demselben Abo-Verhalten wie EIS, Hallenspiegel und Feinplanung (Topics, QoS, Client-ID, Session-Einstellungen), die jede Meldung mit t_{recv} protokollieren [prüfen, ob die echten Verbraucher eingesetzt werden können]	keine, da sie die Verbraucher ersetzen
M6	Testrunner protokolliert Lastplan sowie Beginn, Ende, Ort und Art jedes Fehlers	keine

6.3 Ablauf eines Laufs im Testrunner

1. Umgebung mit leeren Volumes neu starten, Variante und QoS über die Konfiguration setzen.
2. Warten, bis alle Verbraucher verbunden sind und der Mitleser an M4 Meldungen empfängt.
3. Simulator mit Lastplan und Startwert starten, Aufwärm- und Referenzphase abwarten.
4. Fehlerzyklen ausführen: Beginn zufällig wählen, Fehler über die Toxiproxy-API oder `docker kill/stop/pause` auslösen, nach der Fehlerdauer aufheben, Erholung abwarten und alles mit Zeitstempel in das Fehlerprotokoll schreiben.
5. Last stoppen und warten, bis [30 s] lang keine neue Meldung mehr ankommt.
6. Logs aller Sonden und den Datenbankexport unter der `run_id` ablegen.
7. Plausibilität prüfen: Die Anzahl der Meldungen an M6 muss N entsprechen, sonst wird der Lauf verworfen.

6.4 Auswertung

Die Protokolle aller Sonden werden über (`run_id`, `machine_id`, `seq`) zu einer Tabelle mit einer Zeile je Meldung zusammengeführt, die für jeden Messpunkt den Beobachtungszeitpunkt und die Anzahl der Beobachtungen enthält. Daraus ergeben sich Verlust, Duplikate und Latenz je Stufe direkt. Die Zustände $z_j(t)$ werden auf einem Zeitraster von [100 ms] berechnet. Daraus folgen Verfügbarkeit, Ausfälle und RTO. Die Auswertung erfolgt in Python mit pandas und SciPy.

Für die statistische Auswertung [5] liefert jeder Lauf einen Wert je Messgröße. Über die Läufe einer Kombination werden Median und 95-%-Konfidenzintervall angegeben, bei Perzentilen wie dem p99 der Latenz mit Bootstrap. Anteile wie die folgenlosen Fehler werden über alle Fehlerzyklen mit Wilson-Intervall angegeben [4]. Die Varianten werden paarweise mit dem Mann-Whitney-U-Test verglichen, da keine Normalverteilung angenommen werden kann. Als Effektstärke wird Cliffs Delta angegeben [6]. Bei mehreren Vergleichen wird das Signifikanzniveau von 0,05 nach Holm korrigiert.

6.5 Prüfung des Messaufbaus

Bevor Ergebnisse ausgewertet werden, wird geprüft, ob die Messung selbst korrekt ist:

- **Nullmessung:** Ein Lauf ohne Fehler muss $L = 0$ und $D = 0$ ergeben. Andernfalls liegt der Fehler in der Messung oder im Aufbau.
- **Bekannte Fehler:** Der Simulator lässt gezielt einzelne Sequenznummern aus und sendet andere doppelt. Die Auswertung muss genau diese Meldungen als Verlust bzw. Duplikat erkennen.
- **Rückwirkung:** Latenz und CPU-Last des API-Servers werden mit und ohne Mitleser an M4 verglichen.
- **Zeitauflösung:** Alle Zeitstempel stammen von derselben Uhr des Hosts. Das Raster von [100 ms] ist die untere Grenze für RTO und Ausfallzeiten.

7 Ergebnisvorlage

Die folgende Tabelle zeigt, in welcher Form die Ergebnisse berichtet werden. Sie enthält noch keine Messwerte.

Variante	W1: RPO	W1: RTO	T3: Δ	Datenalter p99	Latenz p99
A: bestehende Kette	[–]	[–]	[–]	[–]	[–]
B: gehärtete Kette	[–]	[–]	[–]	[–]	[–]
C: Stern	[–]	[–]	[–]	[–]	[–]

Aus dieser Tabelle ergibt sich die Abwägung zwischen Zuverlässigkeit (RPO, RTO, Δ, Datenalter) und Aufwand (Latenz, Umbau), die die Grundlage der Empfehlung für den weiteren Ausbau bildet.

Literaturverzeichnis

[1] *ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Organization for Standardization, 2023.

[2] *ISO/IEC 25023:2016 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Measurement of system and software product quality*, International Organization for Standardization, 2016.

[3] A. Basiri u. a., „Chaos Engineering,“ *IEEE Software*, Jg. 33, Nr. 3, S. 35–41, 2016. DOI: 10.1109/MS.2016.60.

[4] E. B. Wilson, „Probable Inference, the Law of Succession, and Statistical Inference,“ *Journal of the American Statistical Association*, Jg. 22, Nr. 158, S. 209–212, 1927.

[5] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. New York: Wiley, 1991.

[6] N. Cliff, „Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions,“ *Psychological Bulletin*, Jg. 114, Nr. 3, S. 494–509, 1993.